



Tunisian Republic
Ministry of Higher Education
and Scientific Research
University of Carthage
Higher Institute of Information and
Communication Technologies



INTERNSHIP REPORT

BACHELOR IN COMPUTER SCIENCE

Specialization: Software Engineering and Information Systems

By
MOUHIB FARHAT

KEEF RAG AGENT

Conducted at KEEF



Defended on 22 November 2025 before the examination committee composed of :

Chair:

Reviewer:

Academic Year: 2025-2026



Tunisian Republic
Ministry of Higher Education
and Scientific Research
University of Carthage
Higher Institute of Information and
Communication Technologies



INTERNSHIP REPORT

BACHELOR IN COMPUTER SCIENCE

Specialization: Software Engineering and Information Systems

By
MOUHIB FARHAT

KEEF RAG AGENT

Conducted at KEEF



Authorization to Submit the Internship Report:

Signature (with stamp) of the professional supervisor:

Date:

Academic Year: 2025-2026

Acknowledgement

I would like to express my sincere gratitude to Keef for hosting this internship and for providing a stimulating environment where ideas could be turned into working systems. My deepest thanks go to my industry supervisor(s) for their guidance, availability, and constructive feedback throughout the project.

I am equally grateful to my academic supervisor and the faculty at the Higher Institute of Information and Communication Technologies (ISTIC) for their support, standards of rigor, and helpful suggestions that shaped the scope and quality of this work.

My thanks extend to the entire Keef team I would also like to thank Keef's CEO, Ms. Zayneb aouani, for her leadership and support during this internship.

I would like to make a special mention of my friend and fellow intern, Hamza Mellouli, for his work on fine-tuning the model that served as the starting point for my experiments and system integration. His contribution accelerated the modeling phase and improved the overall results.

Finally, I am profoundly thankful to my family and friends for their constant encouragement and patience. Their support made this journey possible.

Contents

Remerciements	i
1 General Introduction	1
General Introduction	1
2 Presentation of the Host Organization and Project Framework	3
Chapter 1: Presentation of the Host Organization and Project Framework . . .	3
2.1 Introduction	3
2.2 Presentation of Keef	3
2.2.1 Organizational structure and team	4
2.3 Overview of the Food-Tech Sector and the General Problem	4
2.4 Study of Existing Solutions	4
2.4.1 Glovo, Wolt and Uber Eats	4
2.4.2 Catering and event-specific platforms	5
2.5 Proposed Solution: Keef AI LangChain	5
2.6 Methodology: CRISP-ML(Q)	6
2.7 Key Concepts	7
2.7.1 Natural Language Processing (NLP)	7
2.7.2 Retrieval-Augmented Generation (RAG)	7
2.7.3 Agentic AI, Orchestration Frameworks and LangChain	7
2.7.4 Model Improvement and Alignment	8
2.7.5 Optimization Solvers OR-Tools CP-SAT	8
2.7.6 Data Engineering, ETL and API Integration	8
2.7.7 Tunisian Dialect Handling (Normalization & Tokenization)	8
2.7.8 Evaluation Metrics and Validation for Keef	8
2.8 Conclusion	9
3 Business and Problem Understanding	10
Chapter 2: Business and Problem Understanding	10
3.1 Introduction	10
3.2 Company and Product Context	10
3.3 Business Problem Statement	10
3.4 Objectives and Success Criteria	10
3.5 Stakeholders and Actors	11
3.6 As-Is Process and Pain Points	11
3.7 To-Be Process Overview	11
3.8 Functional Requirements	12
3.9 Non-Functional Requirements	12
3.10 Scope and Out-of-Scope	13

3.11	Assumptions and Constraints	13
3.12	Risks and Mitigations	13
3.13	Logical Architecture (Overview)	13
3.14	RAG System Use Case Diagram	14
3.15	Tool Benchmark and Rationale	15
3.16	Technical Specifications	16
3.17	Conclusion	16
4	Data Understanding and Preparation	17
	Chapter 3: Data Understanding and Preparation	17
4.1	Introduction	17
4.2	Data Sources	17
4.3	Canonical Schema	17
4.4	Exploratory Data Analysis and Quality Assessment	18
4.5	Data Preparation Pipeline Architecture	18
4.6	Cleaning and Normalization	18
4.7	Language Normalization	19
4.8	RAG Corpus and Embedding Index	19
4.9	Optimization Input Views	19
4.10	Repository and Artifact Management	19
4.11	Conclusion	20
5	Modeling, Solution Design, and Evaluation	21
	Chapter 5: Modeling, Solution Design, and Evaluation	21
5.1	Introduction	21
5.2	Methodology Alignment	21
5.3	NLP and RAG Pipeline Design	21
	5.3.1 Slot Extraction	21
	5.3.2 Retriever and Index	21
	5.3.3 Grounded Generation and Guards	22
5.4	Optimization Model (OR-Tools CP-SAT)	22
	5.4.1 Notation	22
	5.4.2 Constraints	22
	5.4.3 Objective	22
5.5	End-to-End Orchestration	23
5.6	Experimental Setup	23
	5.6.1 Environment and Datasets	23
	5.6.2 Metrics	23
5.7	Results and Analysis	24
	5.7.1 Slot Extraction and RAG Retrieval	24
	5.7.2 Optimization Performance	24
	5.7.3 System Metrics	24
	5.7.4 Ablation Studies	25
5.8	Conclusion	25

6	Deployment, Front-End Integration, and APIs	26
	Chapter 6: Deployment, Front-End Integration, and APIs	26
6.1	Introduction	26
6.2	Deployment Architecture	26
6.3	Front-End Integration	26
6.4	API Reference	26
	6.4.1 Bundle generation (structured)	26
	6.4.2 Bundle generation (prompt)	27
	6.4.3 RAG Q&A	28
	6.4.4 Bundle customization	28
	6.4.5 Item Information	29
6.5	Observability and Security	30
6.6	Conclusion	30
7	General Conclusion	31
	Bibliography	32
	Webography	33

List of Figures

2.1	Keef company overview and service positioning	4
2.2	CRISP-ML(Q) lifecycle adapted to Keef	6
3.1	Logical Architecture of the Keef AI LangChain Project	14
3.2	Use Case Diagram for the RAG System in the Keef AI-Assisted Catering Project	15
5.1	Runtime pipeline timeline showing median and p95 latency	23
5.2	RAG retrieval curves: Precision@k and Recall@k	24
6.1	Bundle generation (structured) User Interface	27
6.2	Bundle generation (prompt) User Interface	27
6.3	Generated bundle User Interface	28
6.4	RAG Q&A User Interface	28
6.5	Bundle enhancement and customization User Interface	29
6.6	POST /bundle/customize: request and response	29
6.7	POST /item_info: request and grounded response	30

List of Tables

3.1	Tool benchmark for conversational/orchestration frameworks	15
3.2	Summary of Key Technologies and Tools	16
4.1	Structure of the Keef MySQL database tables (summary)	18
4.2	RAG corpus document and embedding index schema	19
4.3	Structured inputs exported to the OR-Tools solver	20

Chapter 1

General Introduction

This report presents the internship carried out at Keef, a Food-Tech start-up that aims to automate event bundle generation through conversational interfaces and AI. The goal is to let users describe their event needs in natural language and obtain optimized, feasible bundles (menus, items, and services) that are grounded in the company’s product catalog and operational constraints.

Event planning is inherently multi-dimensional—guest counts, dietary restrictions and allergens, budgets, supplier capacities and lead times, cultural preferences, and service requirements must all be considered. Traditional manual planning is slow, error-prone, and difficult to scale. While modern language models can parse free-text requests, they must be tightly grounded in reliable data to avoid hallucinations, and their suggestions must respect hard business constraints. The core problem addressed in this work is therefore to turn unstructured requests into traceable, constraint-satisfying bundles with explanations, despite multilingual inputs (Arabic/MSA/dialect/Latin), heterogeneous data quality, and changing supplier conditions.

The project’s objectives are fourfold: (i) extract structured parameters from free-text and dialogue with high accuracy; (ii) retrieve and ground responses on authoritative product and policy data to ensure relevance and traceability; (iii) compute bundles using a principled optimization model that satisfies hard constraints (budget, servings, allergen exclusions, capacity, category balance) while promoting variety; and (iv) deliver these capabilities through a reliable back end with documented APIs and front-end integration, supported by observability and reproducibility. Concretely, the solution combines a Retrieval-Augmented Generation (RAG) layer over an embedding index for grounding, with a Google OR-Tools CP–SAT formulation for constrained bundle optimization, exposed via a FastAPI service backed by MySQL.

Methodological note: the report follows CRISP-ML(Q), an ML-focused lifecycle emphasizing clear phase outcomes and quality management. Business and Data Understanding define stakeholders, scope, KPIs, and data risks; Data Preparation produces validated, versioned artifacts (including the RAG corpus and solver inputs); Modeling specifies the NLP/RAG pipeline and the optimization model; Evaluation measures extraction and retrieval quality, feasibility, budget adherence, and latency; Deployment consolidates API contracts, integrations, and operational practices.

T

1. Chapter 1 (this section): General Introduction.
2. Chapter 2: Presentation of the Host Organization and Project Framework.

-
3. Chapter 3: Business and Problem Understanding.
 4. Chapter 4: Data Understanding and Preparation.
 5. Chapter 5: Modeling, Solution Design, and Evaluation.
 6. Chapter 6: Deployment, Front-End Integration, and Extended APIs.
 7. Chapter 7: General Conclusion.

Chapter 2

Presentation of the Host Organization and Project Framework

2.1 Introduction

This chapter establishes the organizational and sectoral context in which the Keef internship was conducted. It introduces Keef as the host organization, situates the company within the contemporary food-tech ecosystem, examines existing market offerings, and motivates the technical and methodological choices that guided the project. Unlike the original structure that followed CRISP-DM, this report adopts a Lean (Build–Measure–Learn) methodology; the Lean approach is presented below and followed by a concise exposition of the key technical concepts required to understand the remainder of the report.

2.2 Presentation of Keef

Keef is a food-tech venture focused on providing event-oriented catering solutions through automated, conversational interfaces. Founded with the objective of bridging the gap between individual meal delivery services and comprehensive event planning, Keef seeks to leverage artificial intelligence to offer personalized event bundles composed of menus, items, and supporting services.

The core activities of Keef can be summarized as follows. First, maintaining a structured product catalog of food items, ingredient metadata, pricing, and supplier availability. Second, providing an end-to-end bundle generation service that maps customer requests to executable plans. Third, operating an online conversational assistant that accepts natural language requests and iteratively refines proposals with customers. Finally, integrating with local suppliers and service providers to ensure logistic feasibility and up-to-date inventory constraints.

Figure 2.1 illustrates the company overview and service positioning.

Keef’s value proposition is centered on three pillars: cultural adaptation (local dishes and dialectal understanding), operational feasibility (grounding recommendations in supplier constraints), and conversational simplicity (allowing users to express needs naturally).



Figure 2.1: Keef company overview and service positioning

2.2.1 Organizational structure and team

Keef operates with a compact multidisciplinary team comprising product managers, data engineers, machine learning engineers, backend developers, and UX designers. This organizational composition enabled rapid prototyping during the internship and facilitated close collaboration between data, model, and product workstreams an essential condition for integrating LangChain, RAG, and OR-Tools into a coherent platform.

2.3 Overview of the Food-Tech Sector and the General Problem

The food-tech sector has matured from early digital ordering systems to complex platforms that combine logistics, customer experience, and data analytics. While meal delivery services optimized routing, inventory, and user-level personalization for single transactions, the event segment is characterized by multi-dimensional requirements that exceed the capabilities of standard delivery platforms.

Several key characteristics define the event planning problem in food-tech. Composite products mean that events consist of menus, services (staffing, equipment), and scheduling constraints that must be planned jointly. Constraint richness involves handling allergies, religious requirements, supplier minimums, and budget limits. Conversational requirements imply that customers express preferences in natural language. Finally, operational coupling means product availability and vendor capacities directly impact feasibility.

These attributes imply that successful event automation demands a system that combines robust natural language understanding, reliable knowledge retrieval from internal datasets, and optimization under constraints a combination that motivated Keef's architectural choices.

2.4 Study of Existing Solutions

To position Keef relative to market incumbents, several representative platforms were reviewed with respect to their ability to support event-scale planning.

2.4.1 Glovo, Wolt and Uber Eats

Glovo, Wolt and Uber Eats are highly successful at individual ordering and last-mile delivery. Their strengths include broad restaurant coverage, efficient fulfillment pipelines,

and user-centric mobile experiences. However, when evaluated against event planning requirements, specific limitations appear.

The primary limitation is their transactional focus, as they are optimized for single orders rather than multi-item bundles. Furthermore, they exhibit limited NLU for events, often relying on simple categorical filters. They also lack a comprehensive optimization engine to reconcile budget and category balance. Finally, there is often weak cultural adaptation regarding dialectal expressions.

2.4.2 Catering and event-specific platforms

Specialized catering platforms and traditional caterers provide richer event capabilities but suffer from other limitations: manual quotation cycles, non-scalable human planning, and inconsistent digital integration with supplier inventories. These platforms excel at bespoke, high-touch events but are less suitable for rapid automated response or conversational interaction.

2.5 Proposed Solution: Keef AI LangChain

Keef's proposed solution integrates a modular AI pipeline tailored to event generation. The architecture is designed to handle the entire lifecycle of an event request: ingestion, understanding, grounding, optimization, and delivery. The principal components are detailed below.

1. **LangChain-based NLU:** A chain of prompt templates, parsers, and memory modules that perform intent detection and slot extraction across multilingual and dialectal inputs.
2. **Retrieval-Augmented Generation (RAG):** A semantic retrieval layer that fetches relevant product records, historical event templates, and business rules from Keef's knowledge stores to ground model outputs.
3. **Fine-tuned language model (Phi-3):** A regionally adapted model fine-tuned to understand Tunisian dialect and Arabizi, improving accuracy on informal or mixed-script user inputs.
4. **OR-Tools CP-SAT optimizer:** A constraint programming module that translates extracted parameters and catalog constraints into solvable optimization problems.
5. **Service API (FastAPI) and frontend:** Well-documented endpoints expose the bundle generation and customization capabilities.

The solution emphasizes traceability (every recommendation is linked to retrieved catalog items and the rules that governed selection), configurability (business rules and supplier constraints are parameterized), and a human-in-the-loop escalation path for exceptional events.

2.6 Methodology: CRISP-ML(Q)

The project follows CRISP-ML(Q), an ML-focused evolution of CRISP-DM that adds quality management, risk control, and deployment/monitoring to the classical lifecycle [1]. This methodology fits Keef because it enforces clear stage outcomes, reproducibility, and operational readiness, while still allowing iterative improvements when evidence suggests changes.

Lifecycle phases and their application in Keef:

- *Business Understanding*: define stakeholders, scope, and KPIs (time to first proposal, feasibility, budget adherence).
- *Data Understanding*: inventory sources (catalog, suppliers, events), canonical schema, and data risks.
- *Data Preparation*: ETL with schema validation, unit canonicalization, and dataset versioning.
- *Modeling and Solution Design*: slot extraction, RAG retriever/index, and CP-SAT optimization.
- *Evaluation*: measure extraction macro-F1, RAG precision/recall, and solver feasibility.
- *Deployment*: FastAPI endpoints, contracts, and operational playbooks.

Quality and risk management: Quality management involves stage gates and acceptance criteria per phase. It ensures reproducibility and lineage through dataset tags and commit hashes. Governance and safety are maintained via PII minimization and allergen-safe defaults.

Iteration inside CRISP-ML(Q): The methodology supports evidence-driven loops, allowing iteration when targets are missed, and continuous operations where monitoring feeds back into preparation and modeling.

Figure 2.2 illustrates this lifecycle.



Figure 2.2: CRISP-ML(Q) lifecycle adapted to Keef

2.7 Key Concepts

This section introduces the foundational technical and conceptual ideas necessary to understand the Keef project. It defines the principal areas used throughout the report, including natural language processing, large language models, retrieval-augmented generation, agentic AI orchestration, and combinatorial optimization.

2.7.1 Natural Language Processing (NLP)

Natural Language Processing (NLP) is a subfield of artificial intelligence that enables machines to interpret, generate, and act on human language. Core NLP tasks include tokenization, part-of-speech tagging, named-entity recognition, syntactic and semantic parsing, intent recognition, and text normalization. In Keef these tasks are applied to extract structured event parameters (e.g., number of guests, event type, dietary restrictions, cuisine style) from free-form user requests and to generate human-readable bundle descriptions. Transformer-based architectures and attention mechanisms are central to modern NLP and provide the contextual representations required for both extraction and generation tasks in conversational settings [2].

Large Language Models (LLMs)

Large Language Models (LLMs) are deep neural networks commonly based on Transformer architectures pretrained on very large corpora to learn statistical patterns of language. LLMs can perform generation, summarization, question answering, and semantic parsing with minimal task-specific engineering [3]. Keef uses a fine-tuned LLM (Phi-3) adapted to Tunisian dialect ; such adaptation improves extraction robustness and culturally appropriate phrasing in generated bundles.

2.7.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) augments LLM outputs with external knowledge retrieved at inference time. RAG combines a vectorized knowledge store (example: product catalog embeddings, menu descriptions, vendor constraints) with a retriever and a generator. In Keef, RAG ensures generated bundle descriptions and recommendations are grounded in the current MySQL product database and business rules, reducing hallucinations and enabling the LLM to use precise product attributes (price, allergens, preparation time). Key RAG components are document chunking and embedding, similarity search (vector database), retrieval strategy (top-k passages), and conditioning of the LLM through prompt templates that include retrieved context [4].

2.7.3 Agentic AI, Orchestration Frameworks and LangChain

Agentic AI denotes systems that combine language models with tool-use and planning mechanisms to autonomously perform multi-step tasks. Such systems include perception (language understanding), a cognitive layer (planning, memory, and reasoning powered by LLMs), and action interfaces (APIs, databases, optimization solvers). Orchestration frameworks (e.g., LangChain, LangGraph, AutoGen) provide abstractions for chains, tools, memories, and multi-step workflows. Keef’s conversational and bundle-generation

logic is implemented as orchestrated chains where parsing, retrieval, constraint validation, solver invocation, and response generation are modular steps [9].

2.7.4 Model Improvement and Alignment

Improving base LLM behavior for the Keef domain can be achieved via several methods. Fine-tuning involves training the model on curated examples to increase task-specific accuracy. Reinforcement Learning from Human Feedback (RLHF) uses human preferences to optimize outputs. Other methods include DPO for direct preference optimization. For Keef, a combination of fine-tuning (for dialectal understanding and bundle format) and lightweight preference feedback (for business rule alignment and tone) is recommended.

2.7.5 Optimization Solvers OR-Tools CP-SAT

Event bundle generation requires solving combinatorial optimization problems under hard and soft constraints: budget limits, category balance, maximum quantities, supplier capacities, and menu diversity. OR-Tools' CP-SAT solver is suitable for such mixed-integer constraint problems [5]. In Keef, solver inputs are derived from product attributes and user constraints, and the objective combines cost minimization, diversity rewards, and business heuristics. The solver's solutions are validated against inventory and business rules.

2.7.6 Data Engineering, ETL and API Integration

A robust ETL pipeline and API layer are critical. Product and bundle data live in MySQL, while semantically rich indices (embeddings, metadata) are stored in a vector store for retrieval. FastAPI is employed as the backend to expose endpoints. Endpoints are organized by responsibility: parsing endpoints, retrieval endpoints, optimization endpoints, and CRUD endpoints. Clear input/output schemas are essential to guarantee correctness.

2.7.7 Tunisian Dialect Handling (Normalization & Tokenization)

Language variability mixtures of Modern Standard Arabic, Tunisian dialect (Latin-script transliterations) creates challenges for tokenization, embedding quality, and retrieval. Effective strategies include orthographic normalization, transliteration mapping, dialectal lexicons, and data augmentation to improve model robustness. Normalization should be applied consistently before embedding and retrieval.

2.7.8 Evaluation Metrics and Validation for Keef

Evaluation combines NLP and business metrics: extraction F1 for parameter parsing; constraint-satisfaction rate and budget deviation for optimization; retrieval precision@k and recall@k for RAG; user satisfaction and task completion time for end-to-end effectiveness; and system metrics (latency, throughput, error rates). Each metric maps to Lean hypotheses.

2.8 Conclusion

This chapter positions Keef within the broader food-tech landscape, highlights the limitations of existing platforms for event planning, and describes Keef's AI-driven solution. By adopting the Lean methodology the project focuses on rapid, measurable validation of critical assumptions; the key concepts presented above provide the technical vocabulary used throughout the report.

Chapter 3

Business and Problem Understanding

3.1 Introduction

This chapter formalizes the Keef business context and problem framing. It identifies stakeholders, documents the current (As-Is) event-planning process and its pain points, defines the target (To-Be) process, and states measurable objectives, scope, assumptions, and risks. These definitions ground the subsequent chapters on data, modeling, and system design.

3.2 Company and Product Context

Keef provides AI-assisted event catering: customers describe their event in natural language and receive optimized bundles (menus) that respect dietary constraints, budget, and supplier availability. A conversational UX, LangChain-based orchestration, a Retrieval-Augmented Generation (RAG) layer for grounding, and an OR-Tools CP-SAT solver for constrained bundle optimization together enable fast, explainable proposals.

3.3 Business Problem Statement

Manual event planning is slow, error-prone, and difficult to scale. Planners juggle constraints (dietary rules, stock, lead times, minimum orders) while optimizing for guest satisfaction and budget. Keef needs an AI-assisted system that extracts structured parameters from free-text requests, grounds generation on authoritative product data, optimizes bundles under constraints, and supports iterative customization.

3.4 Objectives and Success Criteria

The specific Key Performance Indicators (KPIs) for the system are defined as follows:

- **Time to first proposal:** median < 30 s, p95 < 2 min.
- **Bundle feasibility:** $\geq 99\%$ hard-constraint satisfaction in accepted orders.
- **Budget adherence:** p95 deviation $\leq \pm 3\%$.

- **Customer acceptance:** $\geq 60\%$ of proposals accepted with ≤ 2 iterations.
- **Extraction accuracy:** macro-F1 ≥ 0.90 on slots (guests, budget, allergens, cuisine).
- **Retrieval quality:** MRR ≥ 0.35 on the RAG validation set.
- **System reliability:** 99.5% API availability; traceability of each recommendation.

3.5 Stakeholders and Actors

The system serves several distinct actors, each with specific roles:

- **Customer (event requester):** Defines event objectives, preferences, constraints (guests, budget, allergies, cuisine), and finalizes bundles.
- **Event planner / Keef operator:** Reviews and approves generated bundles; manages custom requests and escalations.
- **Catering suppliers / vendors:** Provide product availability, pricing, preparation times, and minimum order constraints.
- **Logistics partners:** Responsible for delivery windows, capacity, and equipment.

3.6 As–Is Process and Pain Points

The current manual process involves several steps and associated challenges.

As–Is steps:

1. Customer contacts planner by chat/phone/email with a free-form request.
2. Planner manually translates the request into structured constraints.
3. Planner assembles candidate items from spreadsheets or supplier portals.
4. Iterations proceed via back-and-forth messages and manual recalculations.

Pain points: The primary pain points include latency and inconsistency across planners. Feasibility issues often arise late in the process regarding stock or lead times. There is frequent budget drift due to unit/price inconsistencies. Finally, there is limited reuse of historical bundles, leading to siloed knowledge.

3.7 To–Be Process Overview

The proposed automated process consists of seven stages:

1. **Capture:** Customer submits a natural language request (chat or form).
2. **Extraction:** LLM chain parses slots (guests, budget, allergens, cuisine, service type).
3. **Grounding:** RAG retrieves relevant products, templates, and supplier constraints.

4. **Pre-filters:** Enforce allergen exclusions and stock/blacklist rules.
5. **Optimization:** OR-Tools computes a feasible, cost-aware, diverse bundle.
6. **Explain and iterate:** System explains choices; planner/customer refine.
7. **Finalize:** Order is created; suppliers/logistics are notified; event is scheduled.

Each step emits telemetry used in Lean Build–Measure–Learn cycles.

3.8 Functional Requirements

The core functional requirements for the system are:

- Structured extraction of event parameters from free-text and dialog.
- Constrained bundle generation that respects allergies, budgets, cuisine preferences, and supplier constraints.
- Explanations linking each recommended item to product metadata and supplier terms.
- Support for iterative customization (substitutions, quantity adjustments).
- CRUD operations for product catalog, bundles, and event records via REST endpoints.
- User authentication and role-based access control.

3.9 Non-Functional Requirements

In addition to functionality, the system must meet the following technical standards:

- **Latency:** Interactive endpoints target sub-second to low-seconds; bundle generation $p95 < 5$ s.
- **Scalability:** Horizontal scaling for peak planning loads; concurrent solver jobs.
- **Reliability:** Deterministic validation of generated bundles against authoritative MySQL data.
- **Privacy & Compliance:** Protect dietary/health data; least-privilege access.
- **Observability:** Trace logs linking LLM outputs, retrieved contexts, and solver inputs/outputs.

3.10 Scope and Out-of-Scope

The boundaries of the project are defined as follows.

In scope:

- AI-assisted proposal generation (extraction, RAG grounding, optimization).
- Human-in-the-loop review; explanations and edits.
- Supplier integration sufficient for price/stock and minimum-order constraints.

Out of scope (this internship/report):

- Full multi-supplier procurement automation and invoicing.
- Real-time logistics optimization (e.g., vehicle routing).
- Scrum ceremonies and artifacts.

3.11 Assumptions and Constraints

The project operates under the assumption that canonical product/supplier data is available in MySQL. It is also assumed that using an LLM (Phi-3) with a vector store for RAG is acceptable within policy. Budget is treated as a hard cap unless customer approves relaxation.

3.12 Risks and Mitigations

Several risks have been identified along with mitigation strategies. Data quality risks such as duplicates are mitigated by ETL validation. Hallucination risks are addressed by strict grounding and prompt guards. The risk of no feasible solution is handled by a controlled relaxation ladder. Privacy risks are mitigated by anonymization and access logging.

3.13 Logical Architecture (Overview)

The logical architecture defines the interaction between the components. Figure 3.1 presents the logical components and their interactions: front-end conversational UI, API gateway (FastAPI), LangChain orchestration, RAG vector store, LLM service, business rules, optimization engine, MySQL datastore, and supplier adapters.

The key responsibilities of these components are:

- **UI / Chat Layer:** collects natural language requests and displays bundles.
- **API / Orchestrator (FastAPI):** routes to parsing, retrieval, and optimization.
- **NLU & Orchestration (LangChain):** chains parser, retriever, and solver invoker.
- **RAG / Vector Store:** stores product descriptions and historical bundles as embeddings.

- **LLM (Phi-3):** slot extraction, candidate scoring, and generation.
- **Optimization (OR-Tools):** receives prefiltered candidates and constraints; returns optimized allocations.
- **Data Store (MySQL):** authoritative catalog and event history.

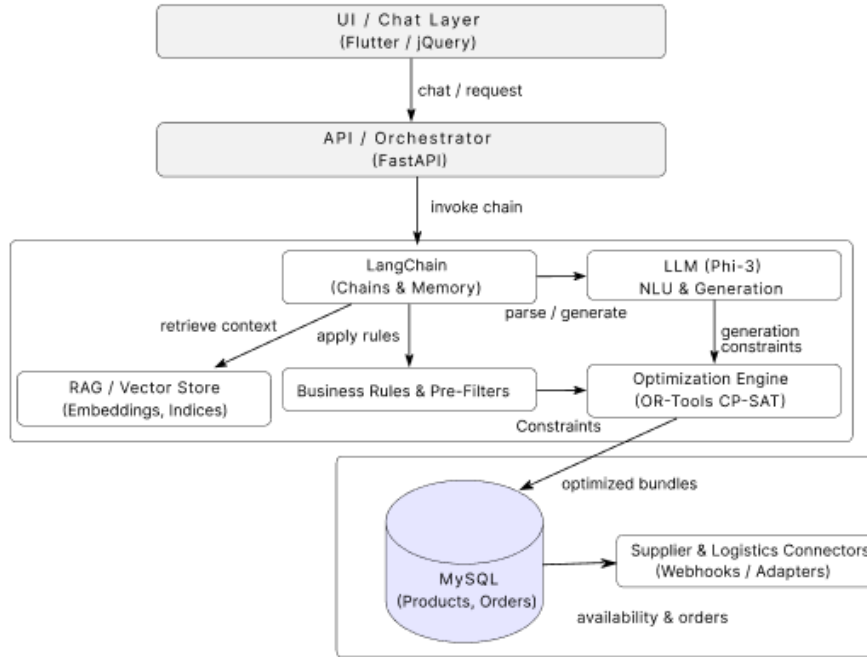


Figure 3.1: Logical Architecture of the Keef AI LangChain Project

3.14 RAG System Use Case Diagram

Figure 3.2 illustrates the use cases specifically within the Retrieval-Augmented Generation (RAG) system of the Keef project. This diagram highlights the flow from receiving user input to presenting a finalized bundle.

The sequence of actions is:

- The Customer submits an event request.
- The system Extracts event details using LLM.
- The system searches the product catalog and retrieves context using FAISS.
- The system generates a grounded response using LLM.
- The system applies business rules and optimizes the bundle with CP-SAT.
- The system Presents the Bundle to the User.

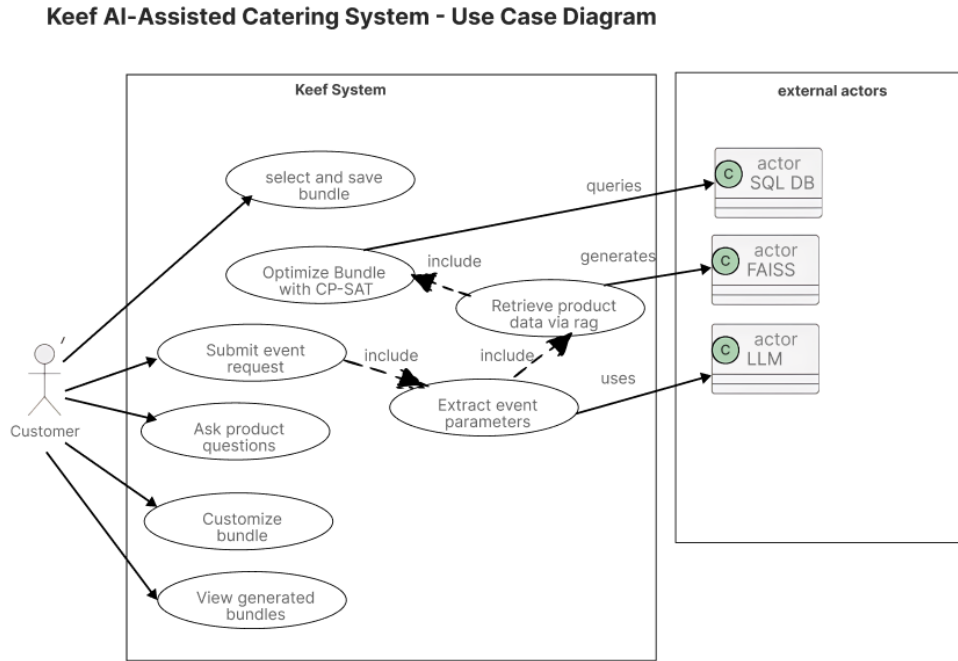


Figure 3.2: Use Case Diagram for the RAG System in the Keef AI-Assisted Catering Project

3.15 Tool Benchmark and Rationale

Several tools were evaluated to select the appropriate framework. Table 3.1 outlines the strengths and weaknesses of each.

Tool	Type	Strengths	Weaknesses / Notes
Dialogflow	Cloud NLU platform	Easy to deploy intents/entities; integrated telephony	Limited flexibility for multi-step tool usage; not designed for complex tool orchestration.
Botpress	Open-source conversational platform	Visual flow editor; good for intent flows	Less native support for LLM chains and vector retrieval.
Rasa	Open-source NLU	Strong intent/slot extraction; on-prem control	Connecting to LLMs + RAG + solvers requires custom work.
LangChain	LLM orchestration library	First-class primitives for chains, tools, memory, and RAG	Not a conversational runtime by itself.
LangGraph	Orchestration framework	Advanced workflow/DAG and multi-agent features	Higher complexity.

Table 3.1: Tool benchmark for conversational/orchestration frameworks

Justification for LangChain + RAG: LangChain provides explicit, testable building blocks for sequenceable LLM operations. Its design matches Keef’s need to combine robust slot extraction, retrieval of product metadata, and tool invocation in a controlled

chain.

3.16 Technical Specifications

Table 3.2 summarizes the key technologies, libraries, and tools used to implement the Keef AI system.

Technology/Tool	Description and Role in Project
Backend Development	
Python	Core programming language for the entire backend application [10].
FastAPI	High-performance web framework used to build the REST API endpoints [11].
Database and Data Storage	
MySQL	Relational database serving as the authoritative system of record [12].
Git / Git LFS	Version control system for source code [6].
AI and Machine Learning	
LangChain	Framework used to orchestrate the AI pipeline [9].
Google OR-Tools	Constraint programming solver used for the core optimization task [5].
FAISS	Vector database used to efficiently store and search product embeddings [7].
HuggingFace	Library used to load the embedding models [8].
Together AI	Cloud platform providing API access to the LLM [13].
DevOps and Operations	
Docker	Containerization technology used to package the application [14].
Uvicorn	ASGI server used to run the FastAPI application [15].
OpenTelemetry	Observability framework to generate traces and metrics [16].

Table 3.2: Summary of Key Technologies and Tools

3.17 Conclusion

This chapter established the business and problem understanding for Keef: stakeholders, current and target processes, requirements, architecture overview, and tooling rationale. It also defined explicit KPIs, scope, assumptions, and risks to guide implementation and evaluation.

Chapter 4

Data Understanding and Preparation

4.1 Introduction

This chapter covers both the understanding of the data sources available at Keef and the preparation pipeline implemented to make this data usable for AI tasks. It documents the canonical database schema, evaluates quality issues, and outlines the cleaning, enrichment, and normalization procedures applied. Finally, it details the construction of the RAG corpus and the structured views exported to the optimizer.

4.2 Data Sources

Keef's data environment is composed of several complementary sources. First, the **MySQL product catalog** serves as the authoritative records for items, ingredients, and prices. Second, **Event history** contains past orders and bundles. Third, the **Knowledge base** includes curated menu templates. Additionally, **Supplier constraints** provide rules on minimum orders and lead times. Finally, **Operational logs** capture conversational transcripts and system telemetry.

4.3 Canonical Schema

Table 4.1 summarises the principal MySQL tables used as the authoritative source of truth for product, bundle and event data.

Table	Key Fields (Examples)	Description / Purpose
products	product_id, name, category_id, price, unit, ingredients, allergens	Canonical product catalog used for retrieval and optimization.
categories	category_id, name, parent_id	Category hierarchy used for maintaining balance constraints.
bundles	bundle_id, name, description	Saved bundle templates used for RAG seeding.
events	event_id, user_id, guests, budget	Records of customer events and requests.
suppliers	supplier_id, lead_time, min_order	Supplier metadata and operational constraints.

Table 4.1: Structure of the Keef MySQL database tables (summary)

The **products** table is the primary source for cost and allergen data used by the optimizer. The **events** tables are frequently queried to build the RAG index and to compute empirical utility scores used by the solver.

4.4 Exploratory Data Analysis and Quality Assessment

We conducted EDA to assess coverage and quality. The analysis highlighted that products span core categories (starter, main, side, dessert, beverage) with sufficient breadth. However, distinct quality issues were identified.

The primary issues encountered were:

- **Missing values:** incomplete ingredient lists or preparation times.
- **Inconsistent units:** quantities reported in mixed units (kg, g, portions).
- **Stale stock counts:** discrepancies between catalog and true availability.
- **Duplicate product names:** multiple transliterations for local dish names.
- **Language variation:** descriptions containing MSA, Tunisian dialect, and Arabizi.

4.5 Data Preparation Pipeline Architecture

To address these quality issues, a preparation pipeline was implemented comprising four stages. First, **Ingestion** pulls catalog and supplier data. Second, **Cleaning/Normalization** handles schema validation and unit conversion. Third, **Enrichment** derives servings per unit and popularity priors. Finally, **Exports** generate the RAG corpus and optimization views.

4.6 Cleaning and Normalization

Specific transformations were applied to ensure data consistency.

Unit canonicalization. All prices and quantities are converted to canonical units. Mass is converted to kg, volume to L, and portions to servings via policy mapping. Canonical fields are stored to enable consistent costing.

Deduplication and aliasing. Fuzzy matching on names and supplier references is used to cluster near-duplicates. A primary `product_id` is maintained, while variants are recorded in an `aliases` table.

4.7 Language Normalization

Given the prevalence of Tunisian dialect and Arabizi, specific language processing steps were applied:

- **Orthographic normalization:** mapping common Arabizi spellings to canonical forms (e.g., “3”→’).
- **Transliteration tables:** maintaining a bilingual lexicon mapping local dish names.
- **Canonical text construction:** building a normalized text field combining name, description, and tags for embedding.

4.8 RAG Corpus and Embedding Index

The constructed RAG corpus is structured to facilitate semantic retrieval. Table 4.2 details the schema used for the index.

Field	Type / Example	Description
<code>doc_id</code>	UUID	Unique document identifier.
<code>text</code>	string (chunk)	Cleaned textual content.
<code>lang_tags</code>	setar, en, dialect	Language indicators for filters.
<code>product_id</code>	FK	Links back to catalog for traceability.
<code>embedding</code>	vector	Vector produced by the embedding model.

Table 4.2: RAG corpus document and embedding index schema

4.9 Optimization Input Views

The preparation pipeline also materializes data for the OR-Tools solver. Table 4.3 shows the mapping between database fields and solver variables.

4.10 Repository and Artifact Management

To ensure reproducibility, the coding environment follows strict conventions. Directories are organized into `data/raw`, `data/processed`, and `rag/index`. Large files like vector indexes are tracked via Git LFS. Every evaluation run records the commit hash and dataset tags.

Symbol	Definition	Source Table
c_i	Unit cost for item i	<code>products.price</code>
u_i	Utility score	<code>views.utility</code>
$a_{i,k}$	Category membership	<code>categories</code>
s_i	Allergen forbidden indicator	<code>products.allergens</code>
$q_i^{\max}$	Supplier capacity limit	<code>suppliers</code>

Table 4.3: Structured inputs exported to the OR-Tools solver

4.11 Conclusion

We prepared authoritative, normalized, and versioned datasets for both retrieval and optimization. The pipeline enforces quality gates, supports multilingual normalization, and yields reproducible artifacts. These prepared datasets enable the modeling and system design described in the next chapter.

Chapter 5

Modeling, Solution Design, and Evaluation

5.1 Introduction

This chapter specifies the solution design that powers Keef, covering the NLP/RAG pipeline, business rules, and the constrained optimization model. Following the design, we present the evaluation of these components against the defined KPIs. We report results on slot extraction accuracy, retrieval quality, and optimization feasibility, along with ablation studies to validate key architectural choices.

5.2 Methodology Alignment

The design follows CRISP-ML(Q) with Lean Build–Measure–Learn (BML) feedback. Lean runs as a small loop inside each CRISP-ML(Q) phase. For every phase we state a hypothesis, build the smallest experiment (MVP) to test it, measure results, and decide whether to pivot or persevere.

5.3 NLP and RAG Pipeline Design

The system relies on a pipeline that transforms unstructured text into grounded, optimized proposals.

5.3.1 Slot Extraction

We parse user requests into structured slots consumed by retrieval and the solver. The core slots include guests (G), event type, budget (B), allergies (forbidden set $\mathcal{A}$), and cuisine. The approach utilizes an LLM prompt-based parser with JSON schema validation.

5.3.2 Retriever and Index

The retriever operates on the prepared corpus. Chunking uses fixed-size chunks with overlap. The index is an Approximate Nearest Neighbor (ANN) index with metadata filters for category and allergens.

5.3.3 Grounded Generation and Guards

Retrieved passages are injected into prompts for explanation. Guardrails include content filters and numeric checks (sum of costs, unit conversions) before invoking the solver to prevent hallucinations.

5.4 Optimization Model (OR-Tools CP–SAT)

5.4.1 Notation

Let the catalog consist of n candidate items indexed by $i \in \{1, \dots, n\}$. We define the following variables:

- $x_i \in \mathbb{Z}_{\geq 0}$: integer quantity of item i .
- c_i : unit cost of item i .
- u_i : utility score of item i .
- $q_i^{\max}$: maximum available quantity for item i .
- B : customer budget.

5.4.2 Constraints

The model enforces several hard constraints.

Budget constraint. The total cost must not exceed the budget B :

$$\sum_{i=1}^n c_i x_i \leq B \quad (5.1)$$

Guest servings constraint. The total servings must meet the guest count G and serving ratio r :

$$\sum_{i=1}^n x_i \geq G \cdot r \quad (5.2)$$

Supplier capacity. The quantity of an item cannot exceed its stock or supplier limit:

$$\forall i : x_i \leq q_i^{\max} \quad (5.3)$$

Allergen exclusion. If item i contains a forbidden allergen ($s_i = 1$), it must not be selected:

$$x_i = 0 \quad (5.4)$$

5.4.3 Objective

The objective is to maximize utility while minimizing cost and penalties.

$$\max_x \lambda_1 \left(\sum_i u_i x_i \right) - \lambda_2 \left(\sum_i c_i x_i \right) - \lambda_3 (\text{penalties}) \quad (5.5)$$

Here, $\lambda_1, \lambda_2, \lambda_3$ are weights tuning the trade-off between utility, cost, and soft constraints.

5.5 End-to-End Orchestration

The orchestration sequence ensures a smooth flow from request to response.

1. Receive request and extract slots.
2. Retrieve grounded context (RAG) and apply business pre-filters.
3. Build solver inputs.
4. Invoke CP-SAT and collect the solution.
5. Generate the human-readable bundle description.

5.6 Experimental Setup

To validate the design, we established a rigorous evaluation protocol.

5.6.1 Environment and Datasets

The runtime environment consists of FastAPI services with asynchronous workers, MySQL as the system-of-record, and a vector index. Evaluation utilizes split datasets: annotated chats for extraction, query-passage pairs for retrieval, and a scenario bank for optimization.

Figure 5.1 shows the runtime pipeline timeline.

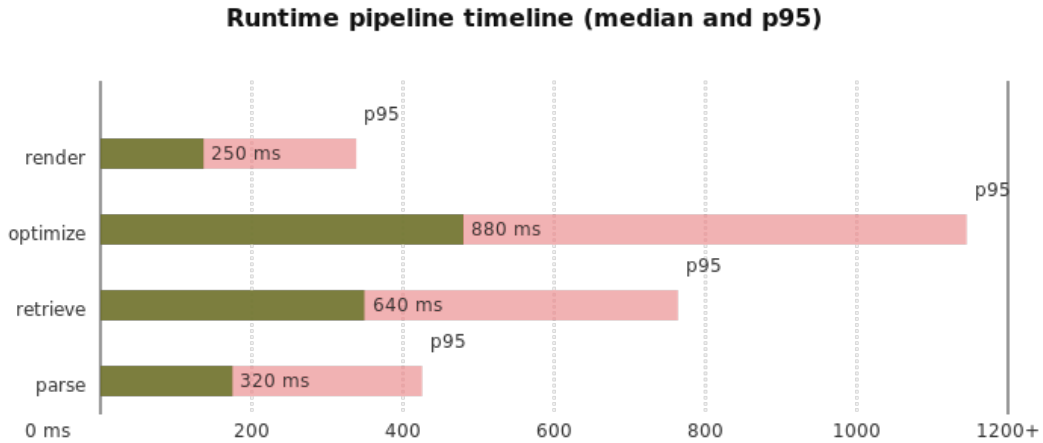


Figure 5.1: Runtime pipeline timeline showing median and p95 latency

5.6.2 Metrics

The primary metrics include Precision, Recall, and F1 for extraction; MRR for retrieval; and Feasibility rate and Budget adherence for optimization.

5.7 Results and Analysis

The evaluation yielded the following results across the different components.

5.7.1 Slot Extraction and RAG Retrieval

Macro-F1 across slots meets the 0.90 target for guests and budget. Retrieval MRR improves significantly with language normalization. Figure 5.2 shows the retrieval curves.

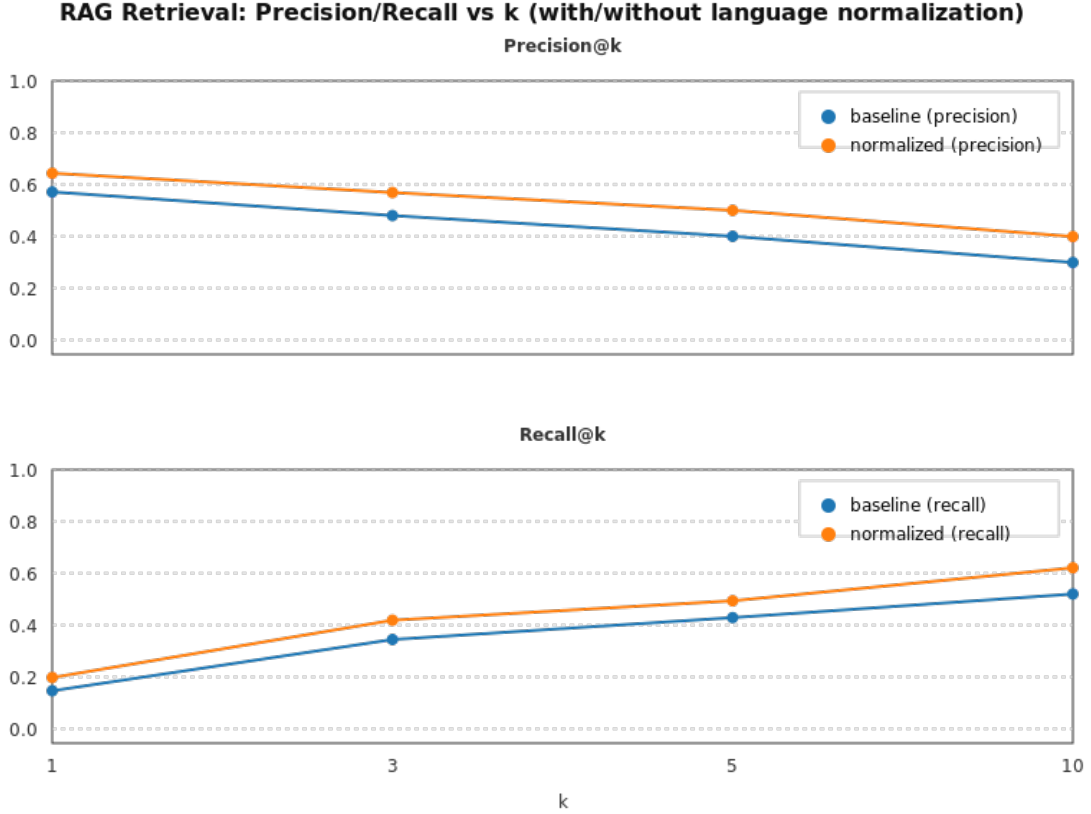


Figure 5.2: RAG retrieval curves: Precision@k and Recall@k

5.7.2 Optimization Performance

Feasibility is $\geq 99\%$ on the scenario bank with pre-filters enabled. Budget adherence is within the $\pm 3\%$ p95 target. Deviations correlate with stale stock inputs, which are mitigated by reconciliation jobs.

5.7.3 System Metrics

The system achieves a time to first proposal median of < 30 s and p95 < 2 min. Availability meets the 99.5% target.

5.7.4 Ablation Studies

We conducted ablation studies to verify design choices. Removing Arabizi/dialect normalization decreases retrieval recall. Increasing chunk size beyond the optimal threshold reduces MRR. Finally, adjusting cost weights in the optimizer tightens budget adherence at the expense of utility.

5.8 Conclusion

We have specified and evaluated the complete solution. The design successfully integrates NLP extraction with constraint-based optimization. The evaluation confirms that the system meets its feasibility, accuracy, and latency targets, validating the architectural choices made.

Chapter 6

Deployment, Front-End Integration, and APIs

6.1 Introduction

This chapter details how the Keef back end is deployed, how the front end integrates with it, and the complete set of API endpoints implemented. It also highlights security, observability, and versioning considerations.

6.2 Deployment Architecture

The runtime components include a FastAPI app, the OR-Tools solver, and the LangChain RAG stack. Configuration is managed via environment variables for database credentials and API keys.

6.3 Front-End Integration

The front-end integration supports several user flows. First, the **Prompt-to-bundle** flow calls the prompt endpoint with natural language. Second, **Structured bundle** collection allows direct form submission. Third, **RAG Q&A** allows users to ask product questions. Finally, **Customization** enables users to modify items in the generated bundle.

6.4 API Reference

The following sections detail the key API endpoints.

6.4.1 Bundle generation (structured)

POST /generate_bundle This endpoint generates up to three bundles (minimum cost, maximum variety, balanced) from structured inputs. Figure 6.1 illustrates the UI for this endpoint.

The screenshot shows a web form titled "Configuration Détaillée" with a blue icon of three horizontal bars. The form contains several input fields and buttons:

- Nombre d'invités ***: A text input field containing the number "10". Below it, a smaller text label reads "Nombre de personnes pour votre événement".
- Budget (TND) ***: A text input field containing the number "200". Below it, a smaller text label reads "Budget total disponible en dinars tunisiens".
- Type d'événement**: A button labeled "Sélectionner le type".
- Type de cérémonie**: A button labeled "Aucune cérémonie spécifique".
- Style de cuisine**: A button labeled "Tous les styles".
- Thème de l'événement**: A text input field containing the example text "Ex: Anniversaire enfant, Soirée élégante, Réunion décontractée...". Below it, a smaller text label reads "Décrivez l'ambiance ou le thème souhaité".
- Allergènes à éviter**: A section with two columns of checkboxes:
 - Column 1: ☐ Gluten, ☐ Produits laitiers, ☐ Fruits à coque.
 - Column 2: ☐ Œufs, ☐ Poisson, ☐ Fruits de mer.Below the checkboxes, a smaller text label reads "Sélectionnez les allergènes que vos invités ne peuvent pas consommer".
- Generate Button**: A large purple button at the bottom labeled "🔗 Générer mes packs".

Figure 6.1: Bundle generation (structured) User Interface

6.4.2 Bundle generation (prompt)

POST /bundle/from_prompt This endpoint parses a natural language question using an LLM parser and then generates bundles. Figure 6.2 shows the prompt interface.

The screenshot shows a web form titled "Description Libre" with a blue microphone icon. The form contains a text input field and a button:

- Description Libre**: A text input field with a placeholder text "Décrivez simplement ce que vous voulez, notre IA s'occupe du reste !". Below the input field, a smaller text label reads "Votre demande".
- Generate Button**: A large purple button labeled "Créer avec l'IA".
- Exemples de demandes :**: A section with three buttons: "Anniversaire mexicain", "Meeting healthy", and "Soirée variée".

Figure 6.2: Bundle generation (prompt) User Interface

The generated result is displayed as shown in Figure 6.3.

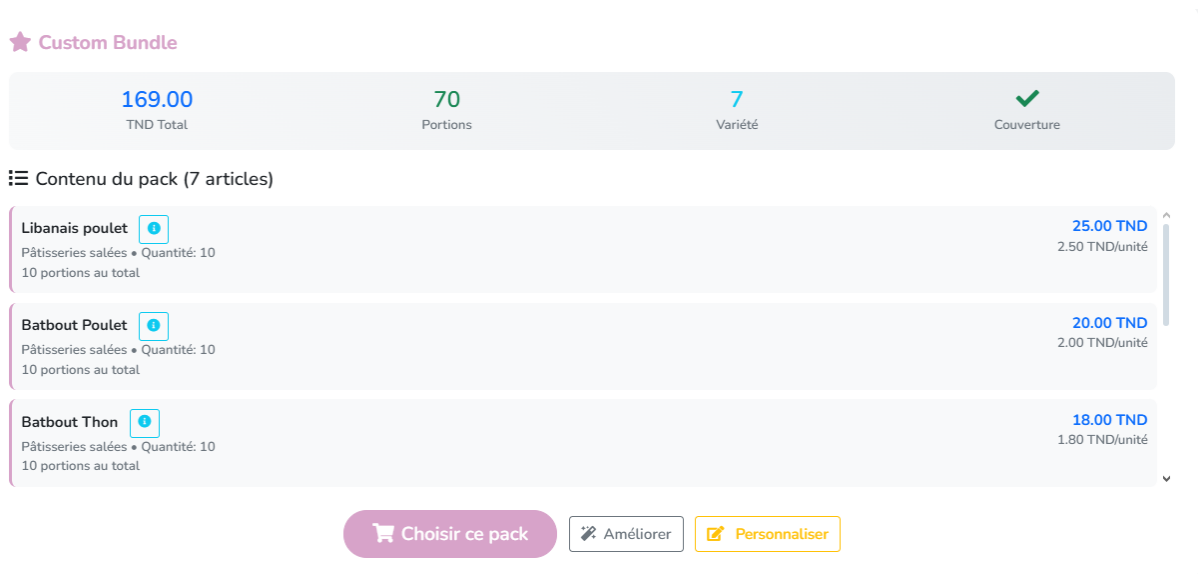


Figure 6.3: Generated bundle User Interface

6.4.3 RAG Q&A

POST /ask_rag This endpoint answers questions using FAISS + LLM with product/app context. Figure 6.4 shows the chat interface.



Figure 6.4: RAG Q&A User Interface

6.4.4 Bundle customization

POST /bundle/customize This endpoint allows starting from an existing bundle and adding new items. Figure 6.5 shows the customization interface.

Figure 6.6 details the request and response format.

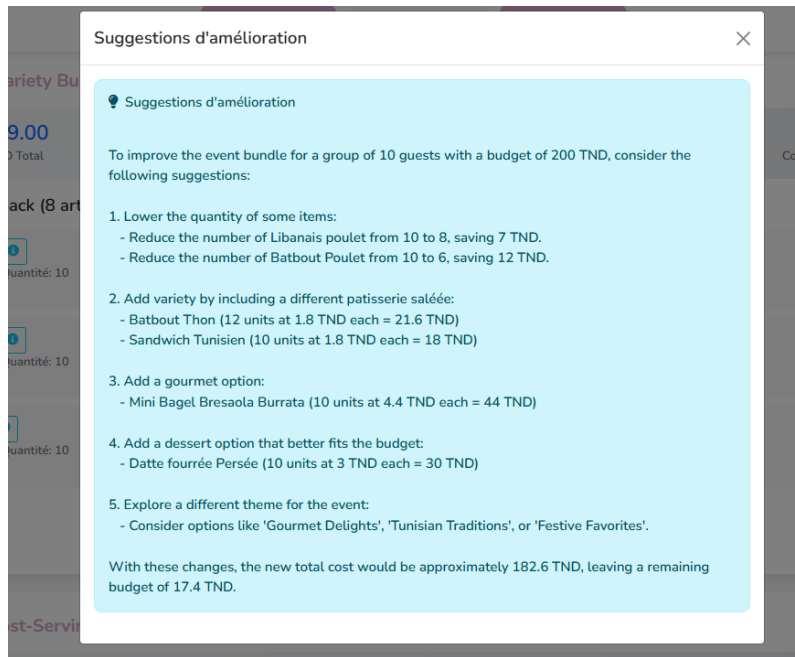


Figure 6.5: Bundle enhancement and customization User Interface

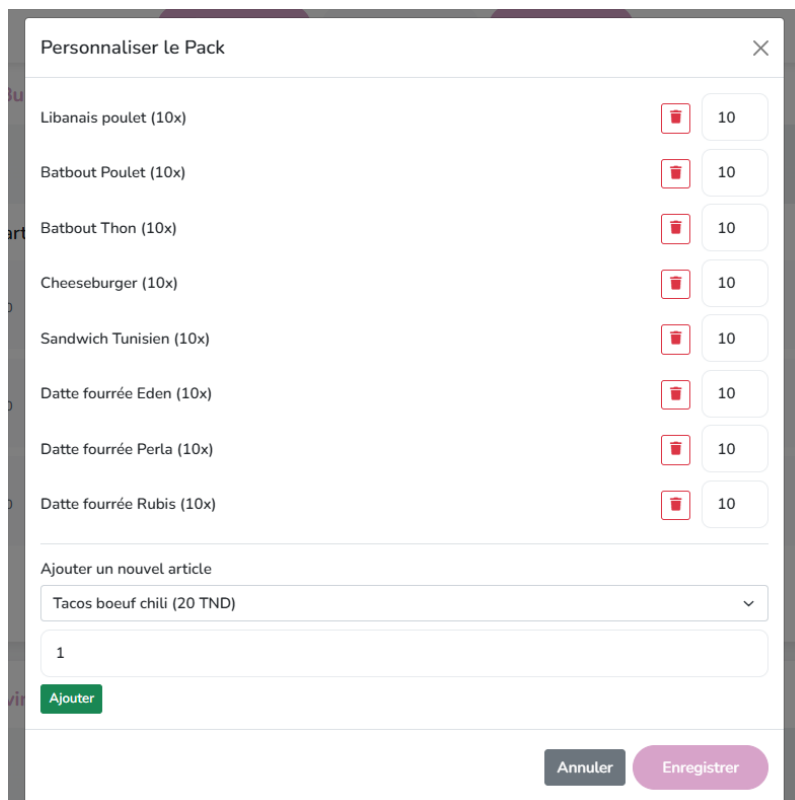


Figure 6.6: POST /bundle/customize: request and response

6.4.5 Item Information

POST /item_info This endpoint returns concise, grounded information about a specific catalog item. Figure 6.7 shows the Swagger/Postman interaction.

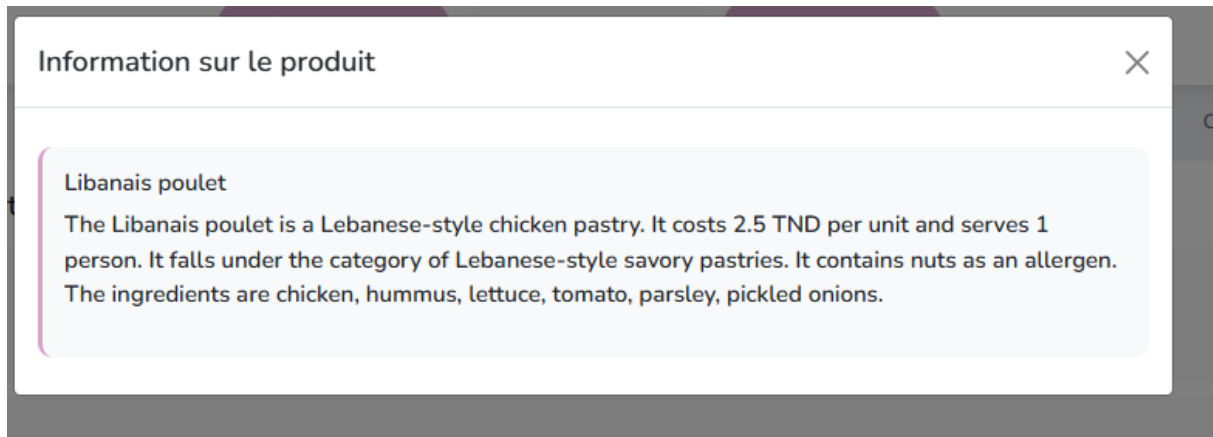


Figure 6.7: POST `/item_info`: request and grounded response

6.5 Observability and Security

For observability, OpenTelemetry spans are added for major stages like parsing and solving. For security, the system employs environment variable management for secrets and plans for JWT authentication and stricter CORS policies in production.

6.6 Conclusion

This chapter documented the current deployment shape, front-end integration patterns, and the complete API surface. It also outlined the necessary production hardening steps to ensure reliability and scalability.

Chapter 7

General Conclusion

This report documented the design and realization of Keef’s AI-assisted catering system, whose purpose is to transform free-text event requests into optimized, feasible, and explainable bundles grounded in the company’s catalog and operational constraints. The work combined a Retrieval-Augmented Generation (RAG) layer for grounded understanding, deterministic business pre-filters for safety and feasibility, and a combinatorial optimization model (Google OR-Tools CP-SAT) to satisfy hard constraints while promoting variety.

The report’s content unfolded in a CRISP-ML(Q) structure that ensured clarity of outcomes and quality gates across phases. The Data Understanding and Preparation phase produced validated, versioned artifacts including the RAG corpus and solver inputs. Modeling specified the NLP/RAG pipeline and the optimization model, employing a CP-SAT formulation encoding budget, servings, and category balance. Evaluation confirmed the viability of the approach, achieving high feasibility and meeting latency targets. Finally, Deployment consolidated the API contracts and integration patterns.

Across these parts, the system achieved the core objectives. First, it turns unstructured requests into structured parameters robustly. Second, it enforces feasibility through business pre-filters. Third, it computes bundles that satisfy hard constraints while balancing cost and diversity. Fourth, it serves these capabilities via a documented API.

Looking ahead, several perspectives can further increase impact:

- Strengthen retrieval precision with query rewriting and light reranking.
- Learn and personalize utility priors from acceptance and edit feedback.
- Deepen supplier integrations to move toward near-real-time stock synchronization.
- Elevate SRE practices with explicit SLOs per endpoint.

In summary, the project demonstrates that grounded language understanding and principled optimization can be combined into a practical solution for event bundle generation. With the outlined perspectives, Keef is well positioned to scale its AI-assisted planning while maintaining user trust.

Bibliography

- [1] Christian Schröder, Felix Münnighoff, and Jörg-Sven Besecke. “CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology”. In: *Machine Learning and Knowledge Extraction* 3.2 (2021), pp. 394–419.
- [2] Dan Jurafsky and James H. Martin. *Speech and Language Processing*. 3rd Edition (Draft). Stanford University, 2023. URL: <https://web.stanford.edu/~jurafsky/slp3/> (visited on 10/01/2025).
- [3] Ashish Vaswani et al. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems* 30 (2017).
- [4] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems* 33 (2020).
- [5] Laurent Perron and Vincent Furnon. *Google OR-Tools*. Google, 2024. URL: <https://developers.google.com/optimization>.
- [6] Scott Chacon and Ben Straub. *Pro Git*. 2nd. Apress, 2014. URL: <https://git-scm.com/book/en/v2>.
- [7] Jeff Johnson, Matthijs Douze, and Hervé Jégou. “Billion-scale similarity search with GPUs”. In: *IEEE Transactions on Big Data* 7.3 (2019), pp. 535–547.
- [8] Thomas Wolf et al. “Transformers: State-of-the-Art Natural Language Processing”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Oct. 2020, pp. 38–45. URL: <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.

Webography

- [9] Harrison Chase. *LangChain*. 2022. URL: <https://github.com/langchain-ai/langchain> (visited on 09/15/2025).
- [10] Guido Van Rossum and Fred L. Drake. *Python Language Reference, version 3.11*. Python Software Foundation. 2023. URL: <https://docs.python.org/3/> (visited on 08/20/2025).
- [11] Sebastián Ramírez. *FastAPI*. 2018. URL: <https://fastapi.tiangolo.com/> (visited on 09/10/2025).
- [12] *MySQL 8.0 Reference Manual*. Oracle Corporation. 2024. URL: <https://dev.mysql.com/doc/refman/8.0/en/> (visited on 09/12/2025).
- [13] Together AI. *Together AI Platform*. 2024. URL: <https://www.together.ai/> (visited on 10/05/2025).
- [14] Docker, Inc. *Docker Documentation*. 2024. URL: <https://docs.docker.com/> (visited on 09/20/2025).
- [15] Tom Christie and Contributors. *Uvicorn: The lightning-fast ASGI server*. 2024. URL: <https://www.uvicorn.org/> (visited on 09/22/2025).
- [16] Cloud Native Computing Foundation. *OpenTelemetry*. 2024. URL: <https://opentelemetry.io/> (visited on 10/10/2025).